

AI AGENTS

Chapter 1: Introduction to AI Agents

By the end you'll be able to tell an agent from a workflow, name the three parts every agent has, and design a tool schema that keeps one in bounds.

MSBA AI Workshop · Session 1 · Sep 18, 2026 · 1:00–3:00 PM · 3039 BIF

Nine Fridays, four parts, one question at the end

Part I · Agents Sessions 1–3	1 Introduction to AI Agents	2 Prompt and Context Engineering	3 Memory, Retrieval and RAG
Part II · Machine learning Sessions 4–5	4 ML Foundations I	5 ML Foundations II	
Part III · Systems and decisions Sessions 6–7	6 Agent Systems	7 When to Use ML, Agents, or Neither	
Part IV · Finance Sessions 8–9	8 Financial Markets and AI as an Investment Theme	9 AI for Financial and Investment Research	

Everything before session 7 is a tool. Session 7 is when you decide which one to reach for.

You are here: session 1, the first of the three agent sessions.

DATES AND ROOMS

Every Friday afternoon, with two exceptions to note now

Part	Date	Time	Room	Topic
Part I · Agents	Sep 18	1:00 – 3:00 PM	3039 BIF	1. Introduction to AI Agents
Part I · Agents	Sep 25	1:30 – 3:30 PM	3007 BIF	2. Prompt and Context Engineering
Part I · Agents	Oct 2	1:30 – 3:30 PM	2043 BIF	3. Memory, Retrieval and RAG
Part II · Machine learning	Oct 16	1:30 – 3:30 PM	3007 BIF	4. ML Foundations I
Part II · Machine learning	Oct 23	1:30 – 3:30 PM	3007 BIF	5. ML Foundations II
Part III · Systems	Oct 30	1:30 – 3:30 PM	3007 BIF	6. Agent Systems
Part III · Systems	Nov 6	1:30 – 3:30 PM	3007 BIF	7. When to Use ML, Agents, or Neither
Part IV · Finance	Nov 13	1:30 – 3:30 PM	3007 BIF	8. Financial Markets and AI as an Investment Theme
Part IV · Finance	Dec 4	1:30 – 3:30 PM	3007 BIF	9. AI for Financial and Investment Research

Today starts at 1:00 in 3039 BIF; every later session is 1:30 in 3007 BIF, except Oct 2 in 2043 BIF. No session on Oct 9, Nov 20 or Nov 27.

WHERE EVERY CHAPTER IS SET

You work at Champaign Capital Research, and it has a problem

An independent equity research firm in Champaign, with a second office in Chicago. It sells written research on companies it covers — and everything it sells is made by hand.

38

people, including two
compliance officers

120

institutional clients
on annual subscriptions

~40

research notes
published a month

3

sectors: industrials, ag
equipment, semiconductors

A firm small enough that one person's afternoon is a real cost — and large enough to matter.

Fictional firm, real mechanics — every workflow you will automate is one a research firm actually runs.

Five colleagues, and one empty desk that is yours

Priya Natarajan

Senior analyst, industrials — covers Deere and Caterpillar (does major in investment research)

Marcus Bell

Associate analyst, supports Priya on industrials

Jordan Lee

Analyst, semiconductors

Elena Ruiz

Head of compliance — clears about 30 staff trades a week

Tom Okafor

Data engineer — vendor feeds and model pipelines

You

Quantitative analyst on the investment team — that seat is yours

Priya owns today's question; Tom owns the feeds the numbers come from.

Nine sessions, one firm, a new problem on your desk each week.

You are the quantitative analyst on the investment team

You

Quantitative analyst

On the investment team, working next to the analysts and Tom's data team. Hired to automate work the firm still does by hand.

- 01 Take a workflow the firm runs by hand and judge what it actually needs.
- 02 Build it — agent, model, or plain code — and keep it inside guardrails.
- 03 Defend the choice to the people whose work it changes.

Nine weeks, nine problems on your desk. Today: the comparison Priya runs by hand.

Every exercise this term is written to you in that seat.

OPEN THIS NOW

The textbook — the code and resources are inside



Scan to open

`dl-msba-workshop.azurewebsites.net`

- Chapter 1 in full, with the agent loop running live in the page.
- The Colab notebook: `notebooks/ch01-agent-loop.ipynb`, runs in the browser.
- Sign in with your Illinois account — the site holds the API key.

Every session has its own chapter here, with that week's code and exercises.

Today: where we're going

01 What an agent is

And when it's the wrong tool

02 Model, tools, loop

Three parts, three failure modes

03 The loop itself

Four lines, and what breaks them

04 Designing the tool API

Schemas beat prompts

05 Deere vs Caterpillar

Four tool calls, one answer

01



What an agent is

The word is doing a lot of work in the market right now. We need a definition precise enough to build against.

THE PROBLEM ON THE DESK

~40



Research notes go out a month, and every one rests on figures somebody pulled and lined up by hand.

Priya covers both Deere and Caterpillar, so comparing them is a question she answers over and over.

Pulling the numbers by hand is slow, and it ages badly

- Revenue, growth and price sit in three different places.
- By the time the note ships, the price line is stale.
- Nothing records which figure came from where, or when.

Hold onto these three — they are what we will judge the agent against.

DEFINITION

AI agent

A system where a language model decides its own next step: it asks for a tool, reads the result, and decides again — until it has an answer or runs out of budget.

WHY IT MATTERS

The model chooses the steps. That is the whole difference, and it is also where the risk lives.

Who picks the next step is the only question that matters

Shape	Who decides	Example
Single call	Nobody — one step	Summarize a document
Workflow	Your code, fixed order	Extract vendor, classify, draft email
Agent	The model, step by step	Compare two companies on growth and price

A workflow is a recipe. An agent is a cook.

Four questions decide whether you need an agent

- Steps known in advance? Use a workflow.
- Steps depend on what you find? Consider an agent.
- What does a mistake cost? Recommend before acting.
- Worth the extra calls, latency, and tokens?

Workflow by default; agent only when the recipe can't be written.

Three calls per trade is cheap; three calls per million rows is not

Good fit

Low volume, high judgment per item.

Steps vary with what's discovered.

A human signs off at the end.

Bad fit

High volume, identical every time.

Steps are already written down.

The action is irreversible.

Today's task sits close to the line — which is exactly why it is worth tracing.

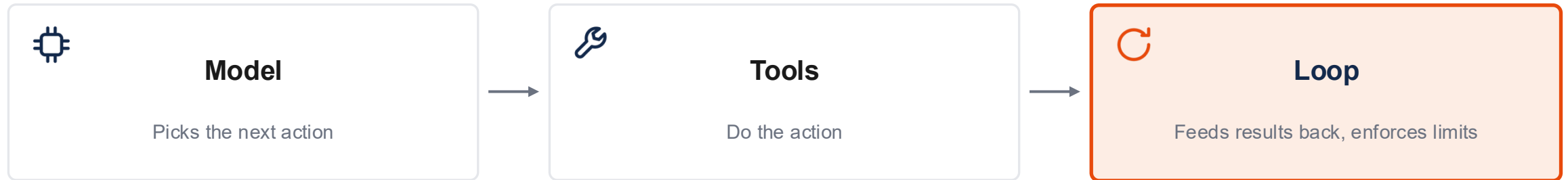
02



Model, tools, loop

Every agent is these three parts. When one misbehaves, knowing which one saves you an afternoon.

The model decides, the tools reach, the loop keeps it honest



Notice, act, check, repeat — the same habit a good compliance officer already has.

The model has no memory — it only sees what you hand it

- Reads the conversation so far, returns a tool call or an answer.
- Nothing carries over between calls.
- Anything it needs must be in the messages.

Wrong answer with the right tools? Look at the model or the prompt.

A tool is a typed contract, not a suggestion

- A name the model can call.
- A description of when to use it.
- A schema listing every allowed argument and its type.

Tool called with nonsense arguments? Look at the schema, not the prompt.

The loop is where your guardrails actually live

- Runs the tool and appends the result to the messages.
- Enforces a step budget and a timeout.
- Writes the log, and holds the approval gate.

A run that never ends? That's the loop, every time.

Read the symptom, and it tells you which part to fix

Symptom	Blame	Fix
Right tools, wrong answer	Model / prompt	Clearer instructions or context
Invalid arguments	Tool schema	Tighten types, enums, required fields
Never finishes	Loop	Step budget and timeout

This table is worth more to you in week three than any prompt-engineering tip.

03



The loop itself

Four lines of logic. Everything a framework adds sits on top of exactly this.

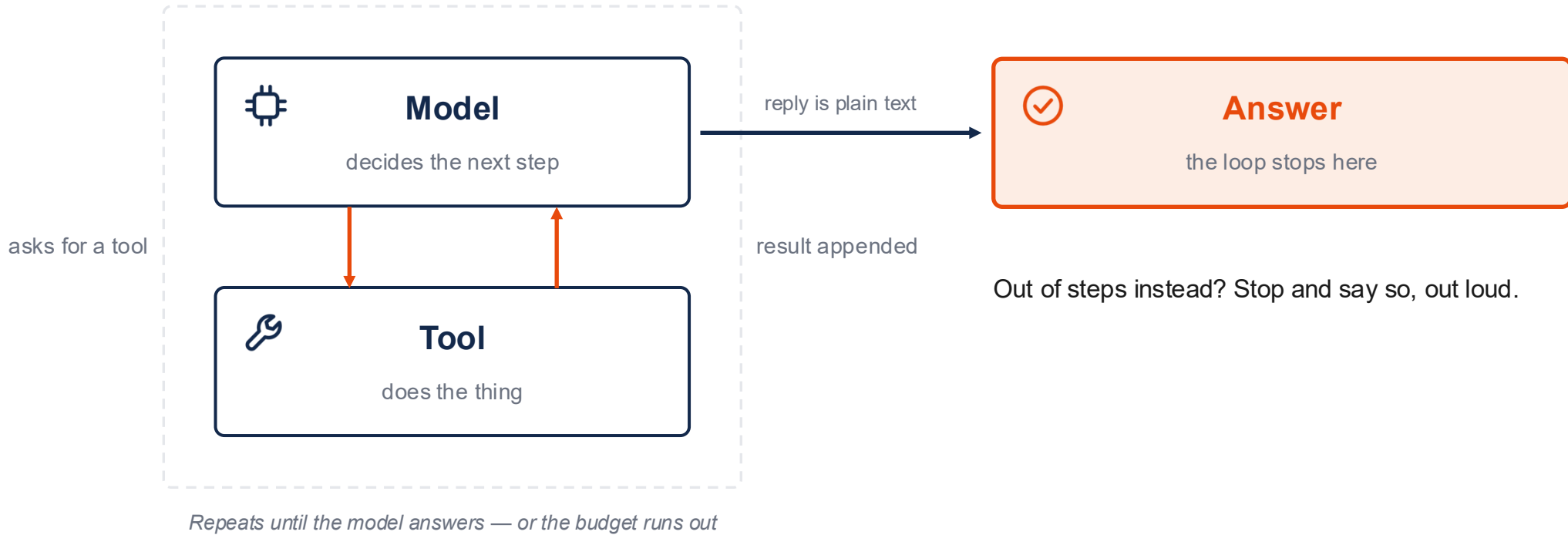
The agent loop, in four lines of plain English

- 1 Send the conversation so far to the model.
- 2 Reply is a tool call? Run it, append the result, go back to step 1.
- 3 Reply is plain text? Stop — that's the answer.
- 4 Out of steps? Stop and say so, out loud.

No hidden machinery. At any moment the conversation is readable end to end.

THE CYCLE

Two boxes and one exit — that is the entire architecture



Every agent framework you will meet is this picture with more logging.

You already work this way — you just do it without naming it

PRIYA, BY HAND

Reads the question.

Decides what she needs next.

Looks it up, notes it down.

Repeats until she can answer.

THE AGENT

≈ Model reads the message list.

≈ Model names a tool and its arguments.

≈ Loop runs the tool, appends the result.

≈ Repeats until the model writes prose.

Same habit, written down as code — with one difference: she remembers, and the model does not.

Which is why the next slide matters more than it looks.

The message list is the model's entire world, and it grows

Call 1

user
the question

Call 2

user
the question

assistant
get_financials(DE)

tool
\$13.8B, +6.4%

Call 3

user
the question

assistant
get_financials(DE)

tool
\$13.8B, +6.4%

assistant
get_financials(CAT)

tool
\$16.9B, +3.1%

Same model, same tools — the only thing that changes is how much it can see.

Each pass is better informed than the last — that is the whole trick.

Two ways a loop goes wrong, and one guard each

Runaway loop

Keeps calling tools, never answers.

Guard: a step budget — six for one question.

Stops loudly: “step budget exhausted.”

Bad call

Calls a tool with invalid arguments.

Guard: catch it, return the error as a result.

The model reads it and corrects itself.

Both guards fail loudly. A silent failure teaches nobody anything.

04



Designing the tool API

The schema is the only hard boundary you have. A prompt asks; a schema refuses.

API

A contract between two pieces of software: you agree to ask in a set way, and it agrees to answer in a set shape.

WHY IT MATTERS

Neither side has to know how the other works. That is what makes it safe to build on somebody else's data.

Two systems, one agreement, data crossing between them



The contract is the only thing they share. Change what is inside either box and nothing breaks — change the contract and everything does.

Same bargain behind every app that shows you a map, a price, or a delivery time.

Every API call is a name, some arguments, and an answer

WHAT YOU SEND

```
get_price(ticker="CAT")
```

The name says which thing. The arguments pin down which one.



WHAT COMES BACK

```
{ "ticker": "CAT", "price": 802.35, "as_of": "2026-09-18 12:01 ET" }
```

Always the same fields, in the same shape.

A tool is an API call the model is allowed to make.

Your browser makes calls like this all day; here the model does the asking.

DEFINITION

Tool schema

The typed contract for one tool: its name, when to use it, and exactly which arguments it will accept — with everything else rejected before it runs.

WHY IT MATTERS

A prompt is advice the model may ignore. A schema is a wall it cannot walk through.

RULE ONE

Use enums wherever a free-text field would do

- A ticker field that accepts strings invites “Deere,” “DE Corp,” “deere.com.”
- An enum of valid symbols forces a real choice.
- Invalid values are refused before any code runs.

Every free-text argument is a guess you agreed to accept.

Mark fields required, and close the schema behind them

- Required means required — even with one valid value today.
- Close the schema so invented arguments are rejected.
- An ignored extra argument is a silent wrong answer.

Tomorrow there will be two valid values, and you'll be glad it was explicit.

RULE FOUR

Saying when not to call a tool beats saying when to

- “Use only when the user asks about revenue, growth, or earnings.”
- Over-calling is the most common failure in a new agent.
- One negative sentence in the description usually fixes it.

Cheaper than a prompt rewrite, and it travels with the tool.

Here is what the four rules look like in one tool

```
"name": "get_financials"  
  
"description": "Revenue and growth only.  
                Do not call for prices."  
  
"ticker":  enum ["DE", "CAT", "NVDA", "AAPL", "MSFT"]  
  
"period":  string, required  
  
"additionalProperties": false
```

- Says when NOT to call it
- Enum — no free text to misread
- Required — forces a real choice
- Closed — invented args rejected

Four rules, one tool, no prompt required.

Read it as a contract: everything not named here is refused.

Three of these four calls never reach your code

Call the model makes	Result	Caught by
ticker: DE, period: Q2-2026	Runs	—
ticker: TSLA	Rejected	Not in the enum
ticker: Deere	Rejected	Not in the enum
... plus currency: EUR	Rejected	Closed schema

Ask the notebook to compare Deere with Tesla, and watch row two happen live.

05



Deere vs Caterpillar, live

One question from Priya, four lookups, and one sentence back. Watch where the agent can still get it wrong.

THE QUESTION

This is the whole task, in one sentence

“Is Deere's revenue growth better than Caterpillar's, and what are both trading at?”

— Priya, 11:40am. The client call is at noon.

Nobody knows in advance whether it takes two calls or four — that is the agent's job.

Two tools, both read-only, both closed

Tool	Takes	Returns
get_financials	ticker (enum of 5), period	Revenue and year-over-year growth
get_price	ticker (enum of 5)	Last traded price

get_financials ships with the notebook; get_price is the one you add in exercise 2.

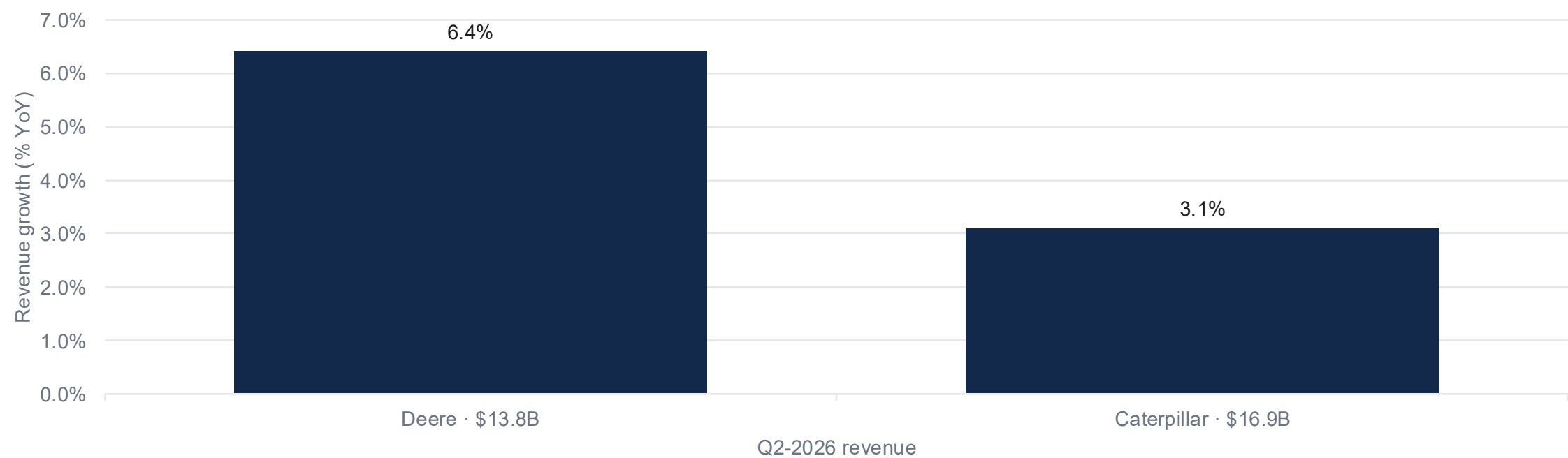
Four tool calls, then one answer

- 1 `get_financials(DE, Q2-2026)` → \$13.8B, up 6.4% on the year.
- 2 `get_financials(CAT, Q2-2026)` → \$16.9B, up 3.1% on the year.
- 3 `get_price(DE)` → \$512.40, then `get_price(CAT)` → \$398.15.
- 4 No tool left to call. The model writes the comparison and stops.

Deere grew faster — 6.4% against 3.1% — on the smaller of the two revenue bases.

THE FINDING

Deere grew twice as fast, on the smaller revenue base



Growth and size point different ways — say which one the client asked about.

Mock figures from the chapter notebook — illustrative, not reported results.

The notebook hands you two tidy quarters. Filings will not.

- The fixture labels both companies Q2-2026.
- In reality Deere's fiscal year runs on its own calendar.
- An agent that calls both “last quarter” is quietly wrong.

Which is why period is an argument at all, rather than something the model infers.

Every figure has to carry where it came from and when

- Which tool returned it, and for which period.
- The as-of time on a price, not just the price.
- A fixture has no as-of time. A real feed does — keep it.

The log is the deliverable; the sentence on top is only the summary.

What to take away

- 01 An agent picks its own steps; a workflow follows yours. Default to the workflow.
- 02 Model, tools, loop — each owns a symptom, so each has its own fix.
- 03 Schemas prevent mistakes that prompts only discourage.
- 04 A figure without its period and its as-of time is not an answer yet.

BEFORE YOU GO

Sign in for attendance



Scan to sign in

`forms.gle/TTWM1rkQYfHxbYmRA`

- Takes about thirty seconds, on your phone.
- Do it now — the form closes after today's session.
- Then open the notebook and run the loop yourself.

Session 2 is Sep 25, 1:30 in 3007 BIF: prompt and context engineering.

Questions?

GO DEEPER

Run the loop yourself: dl-msba-workshop.azurewebsites.net — Chapter 1

Exercise 2 in the notebook: add `get_price` and get the four-call run

Anthropic, “Building effective agents” · OpenAI function-calling, strict schemas



The textbook, and the Colab